CSE HACKATHON DSA - FINAL CODE

Files:
1. problem1_maze.py
   BFS maze solver; saves maze_path.csv.
   Optional visualization: python problem1_maze.py --visualize

2. problem2_closest_pair.py
   Brute force + divide and conquer; checks answers agree.
   Plot closest pair: python problem2_closest_pair.py --visualize
   Benchmark and plot timings: python problem2_closest_pair.py --benchmark

3. problem3_budget_traveller.py
   Dynamic programming with at most K stops and C coupons.
   Saves flight_path.csv.
   Optional graph: python problem3_budget_traveller.py --visualize

4. Problem4ExamScheduling.java
   Greedy DSatur + exact backtracking.
   Saves timetable.csv and conflicts.csv.
   Compile/run:
     javac Problem4ExamScheduling.java
     java Problem4ExamScheduling

5. visualize_problem4.py
   Optional Python visualization for the timetable created by Java.
   Install packages:
     pip install matplotlib networkx pandas
   Then run:
     python visualize_problem4.py

Python dependencies:
- Core algorithms in Problems 1-3 use only Python's standard library.
- Matplotlib is needed for plots.
- NetworkX and Matplotlib are needed for Problem 3 graph visualization.
- Problem 4 visualization helper uses NetworkX and Matplotlib (pandas is not required).

Input formats:
Problem 1:
  N
  N rows; each row can be space-separated cells ("S . a #")
  or a compact row ("S.a#").

Problem 2:
  N
  Generates N random integer coordinate points.

Problem 3:
  N M
  K C
  M lines of: u v price

Problem 4:
  N M R
  M lines of: u v
  Course IDs are 1-based.

Notes:
- In Problem 1, path length means number of moves, not number of cells.
- In Problem 3, at most K intermediate stops means at most K+1 flights.
- In Problem 4, the exact method is exponential in the worst case; use small
  inputs if you need a guaranteed exact answer during a short hackathon.
- The Java exact search starts with the greedy result as an upper bound.
  If it completes, its result is optimal.